

01 — Progressive Web App: come funziona la piattaforma

Come fa una pagina web a **installarsi** come un'app e a **funzionare senza rete**. Il service worker come proxy nel browser, il suo ciclo di vita, la Cache Storage API. Esempi: `sw.js`, `src/app/registra-sw.js`, `manifest.webmanifest`.

1. Una PWA non è una tecnologia, è un insieme di API

«PWA» non è una libreria che si installa né un framework. È un'etichetta per un sito web che usa alcune **API standard del browser** per comportarsi come un'app installata:

- un **web app manifest** — un file JSON che dice al sistema operativo come installare e mostrare il sito (nome, icona, colori, modalità a tutto schermo);
- un **service worker** — uno script che il browser tiene in vita a parte e che può rispondere alle richieste di rete anche quando la pagina è chiusa o non c'è connessione;
- la **Cache Storage API** — un archivio di coppie richiesta/risposta HTTP, su cui il service worker appoggia i file da servire offline.

Niente di tutto questo richiede un passo di build o una dipendenza. È il motivo per cui il vincolo del progetto — *zero build, zero backend* — è compatibile con una PWA completa: sono tutte capacità **native** del browser.

Rispetto ad Angular. Angular ha un pacchetto (`@angular/service-worker`) che *genera* un service worker per te a partire da una configurazione. Qui non c'è generazione: `sw.js` è scritto a mano, riga per riga. Vedere il file vero, senza uno strato che lo produce, è esattamente il punto di questo studio.

2. Il manifest: come il sistema operativo vede l'app

`manifest.webmanifest` è dichiarativo. Il browser lo legge dal `<link rel="manifest">` e, se ci sono i requisiti (HTTPS o localhost, un service worker, delle icone), offre «Installa».

```
{
  "name": "PokéDeck Famiglia",
  "start_url": "./",
  "scope": "./",
  "display": "standalone",
  "theme_color": "#2a5fd6",
  "icons": [ ... ]
}
```

Due campi meritano attenzione tecnica:

- `display: standalone` toglie la barra degli indirizzi: l'app aperta dalla home sembra nativa. È anche la ragione di un problema pratico che ritorna nel §5 — senza barra degli indirizzi **non c'è il pulsante ricarica**.
 - `scope` e `start_url` **relativi** (`./`, non `/`). Un path assoluto `/` ancorerebbe l'app alla radice del dominio; ma su GitHub Pages il sito vive in una sottocartella (`utente.github.io/PokeDeckFamiglia/`). Il path relativo fa funzionare l'app in entrambi i casi. Lo stesso principio vale per **ogni** URL del progetto, service worker compreso.
-

3. Il service worker è un thread separato, senza DOM

Un service worker è un `Worker`: gira su un **thread proprio**, non ha accesso al `document`, al `window`, né al DOM. Non può toccare la pagina. Comunica con essa solo per messaggi (`postMessage`).

Il suo modello di esecuzione è **event-driven ed effimero**: il browser lo sveglia quando c'è un evento (`install`, `activate`, `fetch`, `message`), gli fa eseguire l'handler, e può **spegnerlo** subito dopo. Non è un processo che resta in memoria: non tenere stato in variabili globali sperando di ritrovarlo.

```
self.addEventListener('install', (e) => { /* prepara la cache */ });
self.addEventListener('activate', (e) => { /* pulisci le cache vecchie */ });
self.addEventListener('fetch', (e) => { /* rispondi alle richieste */ });
self.addEventListener('message', (e) => { /* ricevi ordini dalla pagina */ });
```

L'analogia più vicina al tuo background non è Angular ma il lato server: un service worker è come un **filtro servlet** o un **interceptor** che sta *davanti* a ogni richiesta e decide cosa farne — solo che gira nel browser, per il tuo sito soltanto.

Lo scope: perché `sw.js` sta nella radice

Un service worker controlla solo gli URL **al suo livello o più in basso**. Un worker registrato da `/app/sw.js` non potrebbe intercettare `/index.html`. Per questo `sw.js` sta nella radice del progetto e non sotto `src/`:

```
// registra-sw.js - lo scope è la cartella dell'app, non la radice del dominio
navigator.serviceWorker.register(new URL('../sw.js', import.meta.url), {
  scope: './',
  updateViaCache: 'none', // sw.js sempre fresco: è lui ad annunciare le versioni
});
```

4. Il ciclo di vita: `install` → `wait` → `activate` → `controlling`

Qui sta il cuore, e la parte che sorprende di più. Un service worker **non** prende il comando appena registrato. Attraversa stati precisi:

stateDiagram-v2

```
[*] --> Installing: register()
Installing --> Waiting: install ok (parsing + waitUntil)
Waiting --> Activating: skipWaiting() / nessun client con la versione vecchia
Activating --> Activated: activate ok
Activated --> Controlling: clients.claim()
Controlling --> [*]
```

- **Installing** — parte l'evento `install`. È il momento in cui si precarica il «guscio» dell'app. `event.waitUntil(promise)` dice al browser: *non considerare l'installazione finita finché questa promise non si risolve*.

```
self.addEventListener('install', (evento) => {
  evento.waitUntil(async () => {
    const cache = await caches.open(CACHE_GUSCIO);
    // ...precarica ogni file del guscio...
  })();
});
```

- **Waiting** — se una versione precedente sta ancora controllando delle pagine, la nuova **aspetta**. Non si sostituisce a caldo: servirebbe file nuovi a una pagina che ha già in memoria i moduli vecchi, mescolando due versioni dell'app. Il worker nuovo resta in panchina finché tutte le schede con la versione vecchia non si chiudono — **oppure** finché qualcuno chiama `skipWaiting()`.
- **Activate** — quando prende servizio parte `activate`. È il posto giusto per **cancellare le cache vecchie**: ora si è sicuri che nessuna pagina le sta più usando.

```
self.addEventListener('activate', (evento) => {
  evento.waitUntil(async () => {
    const nomi = await caches.keys();
    await Promise.all(
```

```

    nomi.filter((n) => n.startsWith('pokedeck-') && !n.endsWith(VERSIONE))
      .map((n) => caches.delete(n)),
  );
  await self.clients.claim(); // prende il controllo delle pagine già aperte
}());
});

```

- **Controlling** — da qui il worker intercetta le fetch delle pagine nel suo scope. `clients.claim()` gli fa prendere il controllo anche delle schede già aperte, senza ricaricarle.

Il parallelo con onupgradeneeded. Se hai letto 02 — IndexedDB, riconoscerai lo stesso schema: c'è **un solo momento** in cui è lecito toccare l'infrastruttura (il lo schema del database, qui la cache dei file), scatenato da un cambio di versione. Fuori da quel momento, le strutture sono immutabili per il codice che le usa.

Perché il progetto non si aggiorna da solo

`skipWaiting()` è potente e pericoloso: attivarsi subito butterebbe via lo stato della pagina aperta (nel nostro caso, i mazzi appena generati). Perciò `registra-sw.js` **non** lo chiama in automatico. Aspetta che l'utente tocchi «Aggiorna» in una barra, e solo allora manda un messaggio al worker in attesa:

```

// pagina → worker
lavoratore.postMessage({ tipo: 'attiva-subito' });

// worker (sw.js)
self.addEventListener('message', (e) => {
  if (e.data?.tipo === 'attiva-subito') self.skipWaiting();
});

```

Quando il nuovo worker prende il comando scatta l'evento `controllerchange` sul lato pagina, e lì si ricarica — non prima, o si rimetterebbe in piedi la versione vecchia:

```

navigator.serviceWorker.addEventListener('controllerchange', () => location.reload());

```

5. Intercettare le richieste: la Cache Storage API

Due archivi da non confondere:

- la **cache HTTP** del browser, quella di sempre, governata dagli header (`max-age...`). Non la controlli dal codice.
- la **Cache Storage API** (`caches`), un archivio programmabile di coppie `Request` → `Response` che *tu* riempi e svuoti. È questa che rende l'offline.

Nell'evento `fetch`, `event.respondWith(promise)` dice al browser: *non andare in rete come faresti di solito, usa la risposta che ti do io*. Da qui nascono le **strategie di caching**, una per tipo di risorsa:

```

self.addEventListener('fetch', (evento) => {
  const url = new URL(evento.request.url);

  if (url.hostname === 'assets.tcgdex.net') // immagini: scarica una volta, poi cache
    return evento.respondWith(cacheARichiesta(evento.request, CACHE_IMMAGINI));

  if (url.pathname.endsWith('version.json')) // "sei aggiornato?": sempre rete
    return evento.respondWith(reteThenCache(evento.request));

  evento.respondWith(cachePrima(evento.request)); // guscio: prima la cache
});

```

Le tre strategie che vedi in `sw.js`:

Strategia	Quando	Perché
cache-first	guscio dell'app (HTML/CSS/JS), indice dei set	apertura istantanea, offline garantito
cache-on-demand	file dei singoli set, immagini	190 file / 6,4 MB: precargarli tutti = scaricare l'intero catalogo alla prima apertura. Si salvano man mano che servono
network-first	<code>version.json</code>	deve riflettere il deploy corrente; la cache è solo riserva offline

cache-first in essenza:

```
async function cachePrima(richiesta) {  
  const salvata = await caches.match(richiesta, { ignoreSearch: true });  
  if (salvata) return salvata;           // c'è: rispondi al volo, niente rete  
  return fetch(richiesta);              // manca: vai in rete (e magari salvata)  
}
```

La trappola delle risposte *opaque*

Un `<img>` verso un altro dominio (le scansioni su `assets.tcgdex.net`) produce una risposta **opaque**: per sicurezza il browser te la consegna ma non te ne fa leggere niente, e `status` vale 0 — quindi `response.ok` è false **anche quando è andata benissimo**. Se filtrassi con il solo `ok`, nessuna immagine finirebbe mai in cache e offline sarebbero tutte vuote:

```
if (risposta.ok || risposta.type === 'opaque') {  
  await cache.put(richiesta, risposta.clone()); // .clone(): il corpo si legge una volta sola  
}
```

(Rovescio della medaglia: essendo illeggibile, una risposta *opaque* viene salvata anche se in realtà era un 404.)

6. Versionare e pubblicare

Cambiare **una costante** basta a pubblicare una versione nuova:

```
const VERSIONE = 'v19';  
const CACHE_GUSCIO = `pokedeck-guscio-${VERSIONE}`;
```

Il nome della cache contiene la versione: quando `activate` cancella «tutto quello che non finisce con `v19`», le cache vecchie spariscono e restano solo quelle nuove. Un'eccezione voluta: la cache dei **dati** (`pokedeck-dati`) non è versionata, così i set già scaricati sopravvivono a un aggiornamento che magari cambia solo un CSS.

Due dettagli d'installazione che valgono oro (imparati sbagliando, vedi i commenti in `sw.js`):

- `fetch(url, { cache: 'reload' })` durante l'install scavalca la cache HTTP: senza, un file servito con `max-age` verrebbe ripescato vecchio e infilato *dentro* la cache nuova — si installerebbe una versione nuova piena di file vecchi.
- **niente** `cache.addAll()`: è tutto-o-niente e fallisce *in silenzio* se un file è stato rinominato. Un `Promise.allSettled` file per file installa quello che c'è e **avvisa** su quello che manca, invece di lasciare il worker vecchio al comando per sempre.

7. Farsi installare: un evento che non tutti emettono

Il manifest e il service worker rendono l'app *installabile*, ma il browser l'invito lo nasconde in un menu. Serve chiederlo — e qui i browser si dividono in due mondi che non si somigliano.

Chrome, Edge, Android emettono `beforeinstallprompt` quando i requisiti sono soddisfatti. L'evento si può **rimandare**: lo si blocca, si mostra il proprio invito, e quello vero si fa comparire quando lo decide l'utente.

```
window.addEventListener('beforeinstallprompt', (evento) => {
  evento.preventDefault(); // senza, Chrome mostra la SUA barra e la nostra è un doppione
  invitoBrowser = evento; // si mette da parte
  mostra();
});
```

Il pezzo controintuitivo: `evento.prompt()` si può chiamare **solo** in risposta a un gesto dell'utente, e una volta sola. L'evento è un buono spendibile una volta, non una funzione richiamabile a piacere.

Safari su iPhone non lo emette affatto, e non offre nessuna API equivalente: lì l'installazione è solo manuale, da Condividi □ «Aggiungi alla schermata Home». Quindi l'unica cosa utile è **spiegare come si fa**. È il motivo per cui `src/app/installazione.js` ha due modi invece di uno: non è una scorciatoia, è che le due piattaforme offrono cose diverse.

Anche capire se l'app è *già* installata richiede due controlli, perché lo standard e Safari non concordano:

```
window.matchMedia?.('(display-mode: standalone)').matches === true ||
window.navigator.standalone === true // solo Safari, da sempre
```

Il paragone con Angular è interessante: qui non c'è nessun framework che normalizzi le differenze fra browser. `beforeinstallprompt` non è nemmeno uno standard W3C — è una proposta implementata da alcuni motori e ignorata da altri. Scrivere per il web significa ogni tanto scrivere due volte la stessa funzionalità, e **dichiararlo nel codice** invece di far finta che sia una.

`preventDefault()` è una promessa, non un interruttore

Questa è costata un guasto vero: su PC «Aggiungi come app» ha smesso di funzionare.

Chiamare `evento.preventDefault()` significa **prendersi la responsabilità** dell'installazione: da quel momento il browser non la propone più da sé, perché gli hai detto che ci pensi tu. Nella prima versione l'evento veniva poi messo a `null` dopo un rifiuto, e la barra si chiudeva con «Più tardi». Risultato: né la via del browser, né la nostra. L'utente restava senza nessuna strada fino al ricaricamento della pagina.

Due regole ne sono uscite:

1. **L'evento non si butta mai via** per tutta la sessione. È l'unico handle che si ha, e il browser non lo rimanda prima della visita successiva.
2. Se un invito si può chiudere — e deve potersi chiudere — allora da qualche parte serve **un punto d'ingresso che non sparisce**. Qui è il pannello «Installazione» in Regole, che ignora di proposito il «non chiedermelo più»: quella spunta zittisce l'app, non disattiva la funzione.

È un caso particolare di una regola generale: quando prendi il controllo di un comportamento predefinito del browser — `preventDefault` su un submit, su un drop, su una navigazione — stai **sostituendo** qualcosa che funzionava. Se la tua sostituzione ha un percorso in cui non fa niente, hai tolto una funzionalità invece di migliorarla.

Chiedere una volta e poi tacere

Un invito che ritorna a ogni avvio si impara a chiudere senza leggerlo. La spunta «non chiedermelo più» finisce in `localStorage` — come il tema, e per la stessa ragione: è una preferenza, non un dato della collezione.

Due dettagli che sembrano minuzie e non lo sono:

- La spunta si legge **alla chiusura, qualunque pulsante l'abbia causata**. «Installa» + «non chiedermelo più» è una combinazione sensata (installo adesso e non voglio più sentirne parlare), e ignorarla sarebbe una piccola bugia sull'unico comando che l'utente ha per zittire l'app.
 - `localStorage` può **lanciare al solo accesso** in navigazione privata. Il fallimento vale «non ho mai detto di no», che è innocuo; farlo esplodere no.
-

8. Verifica

1. Un service worker registrato da `/js/sw.js` riesce a intercettare la richiesta di `/index.html`? Perché sì o perché no?
2. Spiega con parole tue perché, dopo aver pubblicato una versione nuova, l'utente continua a vedere quella vecchia finché non tocca «Aggiorna». In quale **stato** del ciclo di vita è il worker nuovo, in quel frattempo?
3. `response.ok` è `false`. Da questa sola informazione, puoi concludere che la richiesta è fallita? (Suggerimento: `response.type`.)
4. Perché `version.json` usa *network-first* e non *cache-first* come tutto il resto del guscio? Cosa vedrebbe l'utente se sbagliassi strategia proprio su quel file?
5. **Esercizio.** Le immagini oggi restano in cache per sempre. Abbozza una strategia per tenere solo le ultime *N* immagini viste (una *cache LRU*). Quali API di `caches` ti servono? (Suggerimento: `cache.keys()` restituisce le `Request` salvate, in ordine di inserimento.)